

The nodes.py Node Library

An extended reference to the Python wrappers around Blender Geometry Nodes

Package `geometry_nodes/` • companion file `geometry_nodes_modifier.py`

This document catalogues the node wrapper classes defined in `geometry_nodes/nodes.py` and explains the conventions you must follow to build and read scenes with them. Every wrapper is a thin Python object around a single `bpy` geometry node (or, for the `NodeGroup` family, a whole sub-tree). The goal of the library is to let you describe a node graph as ordinary Python — constructing nodes, passing sockets between them, and chaining their geometry ports — instead of clicking in Blender’s node editor. Read it top to bottom once; thereafter use the catalogue (§3–§11) as a lookup table.

Contents

1 The big picture

- 1.1 Three node “colours”
- 1.2 Sockets you rely on by name
- 1.3 Layout coordinates are grid units, not pixels
- 1.4 Automatic vs hand layout

2 The base Node class

- 2.1 The constructor kwarg pattern (read this twice)

3 Green nodes — geometry I/O

- 3.1 Worked snippet
- 3.2 Primitives & conversions
- 3.3 Curve operations
- 3.4 Mesh editing
- 3.5 Instances
- 3.6 Selection, sorting, splitting
- 3.7 Sampling another geometry
- 3.8 Whole-geometry transforms & info

4 Red nodes — inputs & field readers

- 4.1 Constant inputs
- 4.2 Per-element fields
- 4.3 Mesh topology readers
- 4.4 Rotation algebra (red)

5 Blue nodes — compute & math

- 5.1 Scalar / vector math
- 5.2 Vectors, matrices, conversions
- 5.3 Rotation algebra (blue)
- 5.4 Random & statistics

6 Switches

7 Structural / special nodes

8 Zones — loops & simulation

- 8.1 RepeatZone — fixed iteration count
- 8.2 ForEachZone — iterate over a domain
- 8.3 Simulation — state across frames

9 The make_function RPN builder

- 9.1 RPN operator vocabulary

10 Wiring helpers

11 Custom group nodes — NodeGroup

- 11.1 Catalogue of ready-made groups (NG)

12 Building a scene: GeometryNodesModifier

5

13 XML import path

5

14 Footguns actually hit

6

1 The big picture

1.1 Three node “colours”

Every wrapper inherits from one of three colour classes, and the colour tells you the *shape of its Python API*:

GreenNode (G) Nodes that carry *geometry*. They expose `.geometry_in` and `.geometry_out` so they can be piped together left-to-right with `create_geometry_line(...)`. Almost all mesh/curve/instance operators live here.

RedNode (R) Pure *input / source* nodes. No geometry input. They expose `.outputs` and usually `.std_out` (“the main output socket”). Field readers (`Position`, `Index`, `NamedAttribute`) and the `Input*` constants live here.

BlueNode (B) Pure *compute / math* nodes. They expose `.outputs` and always set `.std_out = outputs[0]`. Math, switches, RNG, type conversion.

A fourth, uncoloured group (**N**) holds structural nodes (`GroupInput`, `GroupOutput`, `Reroute`, `Frame`), and the `NodeGroup` family (**NG**) builds an *entire sub-tree* that appears in the parent graph as one tidy node.

1.2 Sockets you rely on by name

`self.geometry_in` / `.geometry_out` the canonical mesh ports on green nodes.

`self.std_out` “the main output”. Always present on blue nodes; present on most red nodes; present on green nodes only when there is a single obvious output.

`self.inputs` / `self.outputs` direct references to `self.node.inputs/outputs`. Use these whenever the named convenience attribute is not there.

1.3 Layout coordinates are grid units, not pixels

The base `Node.__init__` multiplies the supplied location by `node_width=200` (x) and `node_height=100` (y). So `location=(3,-2)` ends up at (600,-200) in Blender. Half-integer steps are fine.

```
# (1, 0) grid ==> (200, 0) px in Blender
node = SetPosition(tree, location=(1, 0))
```

1.4 Automatic vs hand layout

`GeometryNodesModifier.__init__(automatic_layout=True)` (the default) calls `layout(tree, mode="Sugiyama")` *after* `create_node`, which re-positions everything. For any visually structured graph (substitution graphs, classifiers, anything you will cross-reference with an XML export) pass `automatic_layout=False` and place nodes yourself.

2 The base Node class

Every wrapper follows the same two-step birth: first create the underlying bpy node into `self.node`, *then* call `super().__init__`, which handles location, naming, and parenting.

```
class SetPosition(GreenNode):
    def __init__(self, tree, location=(0, 0),
                 geometry=None, selection=None,
                 position=Vector([0,0,0]),
                 offset=Vector([0,0,0]), **kwargs):
        # 1) create the real bpy node first
        self.node = tree.nodes.new(
            type="GeometryNodeSetPosition")
        # 2) base class: location / name / parent
        super().__init__(tree, location=location,
                         **kwargs)
        ...
```

`Node.__init__` consumes a handful of universal `**kwargs`:

`hide=True` collapse the node visually.

`name="..."` sets both `node.name` and `node.label`.

`label="..."` sets only the label.

`parent=<Node>` re-parents (e.g. into a `Frame`); the location is then interpreted relative to the parent.

2.1 The constructor kwarg pattern (read this twice)

Most wrappers accept a source socket *as* a kwarg and wire it for you — but only when you actually pass it. Pass a literal (`Vector/int/float`) instead and many wrappers set the socket's `default_value`:

```
# socket → creates a link
SetPosition(tree, geometry=grid.geometry_out,
            position=fn.outputs["p"])

# literal → sets default_value
SetPosition(tree, geometry=grid.geometry_out,
            offset=Vector([0, 0, 1]))
```

The typical implementation that produces this dual behaviour:

```
if isinstance(position, (Vector, list)):
    self.node.inputs["Position"].default_value = position
else:
    self.tree.links.new(position,
                        self.node.inputs["Position"])
```

Caveat. Not every wrapper honours its own kwarg. When a kwarg you want is missing (or silently ignored), set the socket directly:

```
s = SampleIndex(tree, index=0)
s.node.inputs["Index"].default_value = 3 # override
```

See the footguns in §14 for the specific offenders (`SeparateGeometry`, `SampleNearestSurface`).

3 Green nodes — geometry I/O

Green nodes are the backbone of every scene: they take geometry in and push geometry out. Chain them with `create_geometry_line`.

3.1 Worked snippet

```
grid = Grid(tree, size_x=4, size_y=4)
del_geo = DeleteGeometry(tree, selection=mask)
set_pos = SetPosition(tree, position=p_fn.outputs["p"])
smooth = SetShadeSmooth(tree)
mat = SetMaterial(tree, material=my_material)

# pipe their geometry ports L→R, ending at the output
create_geometry_line(tree,
    [grid, del_geo, set_pos, smooth, mat],
    out=out.inputs["Geometry"])
```

3.2 Primitives & conversions

MeshLine (G) straight line of vertices; `count`, `start_location`, `end_location`.

Grid (G) XY grid; `size_x`, `size_y`, `vertices_x/y`.

CubeMesh (G) primitive cube.

UVSphere (G) ,

IcoSphere (G) spheres.

CylinderMesh (G) ,

ConeMesh (G) ,

MeshCircle (G) round primitives.

Points (G) *N* free-floating points (no faces).

CurveLine (G) ,

CurveCircle (G) ,

CurveArc (G) ,

CurveQuadrilateral (G) ,

Quadrilateral (G) curve primitives. `Quadrilateral` in `mode="POINTS"` lets you wire `Point 1..4` from sockets.

MeshToCurve (G) ,

MeshToPoints (G) ,

PointsToVertices (G) ,

PointsToCurve (G) ,

CurveToMesh (G) type conversions.

StringToCurves (G) text → curves.

VolumeCube (G) ,

VolumeToMesh (G) volume primitives.

3.3 Curve operations

ResampleCurve (G) `mode="Count"/"Length"/...`

TrimCurve (G) ,

FilletCurve (G) ,

SampleCurve (G) trim, round corners, sample a value/position along a curve.

FillCurve (G) close a curve into a face (`mode="N-gons"`).

3.4 Mesh editing

ExtrudeMesh (G) extrude verts/edges/faces (`mode=`).

SubdivideMesh (G) ,

SubdivisionSurface (G) subdivision.

DualMesh (G) ,

FlipFaces (G) ,

SplitEdges (G) topology edits.

ScaleElements (G) scale faces/edges about their centre.

MergeByDistance (G) ,

SetShadeSmooth (G) cleanup / shading.

ConvexHull (G) ,

BoundingBox (G) (outputs Min, Max, Box).

MeshBoolean (G) union/diff/intersect (operation, solver).

WireFrame (G) ,

CurveWireFrame (G) ,

WireFrameRectangle (G) custom wire-frame builders.

3.5 Instances

InstanceOnPoints (G) ,

InstanceOnEdges (G) scatter instances.

ScaleInstances (G) ,

RotateInstances (G) ,

TranslateInstances (G) per-instance transforms.

GeometryToInstance (G) wrap geometry as one instance.

RealizeInstances (G) flatten instances back to mesh.

```
iop = InstanceOnPoints(tree, points=pts.geometry_out,
                      instance=sphere.geometry_out)
rot = RotateInstances(tree, instances=iop.geometry_out,
                    rotation=rot_fn.outputs["r"])
```

3.6 Selection, sorting, splitting

SeparateGeometry (G) split by selection; outputs .geometry_out (selected) and .inverse. The geometry= kwarg does not auto-link — see §14.

DeleteGeometry (G) delete by selection (mode="POINT"/"EDGE"/"FACE").

SortElements (G) reorder a domain.

DistributePointsOnFaces (G) Poisson/random surface sampling.

3.7 Sampling another geometry

SampleIndex (G) read an attribute at a specific element index.

SampleNearest (G) ,

SampleNearestSurface (G) nearest-element sampling. For SampleNearestSurface the input socket is named "Mesh", not "Geometry" — wire it manually.

GeometryProximity (G) closest position on a target; .std_out = Position.

IndexOfNearest (G) nearest-index lookup.

RayCast (G) cast a ray; outputs hit position, normal, etc.

```
# read a per-face attribute from a *separate* mesh:
named = NamedAttribute(tree, name="region",
                      data_type="INT")
snn = SampleNearestSurface(tree, data_type="INT",
                          sample_position=p.std_out)
tree.links.new(src, snn.node.inputs["Mesh"]) # !
tree.links.new(named.std_out, snn.node.inputs["Value"])
```

3.8 Whole-geometry transforms & info

TransformGeometry (G) apply one transform to the whole geometry.

JoinGeometry (G) merge any number of streams (feed geometry= a list).

DomainSize (G) element counts per domain.

ObjectInfo (G) geometry/transform from a Blender object.

SetMaterial (G) assign a material to a selection.

StoredNamedAttribute (G) write a named attribute on a domain. Kwargs name, data_type, domain, value, selection.

ImportCSV (G) load a CSV as geometry attributes.

```
join = JoinGeometry(tree)
for g in geometries:
    tree.links.new(g, join.geometry_in) # repeatable
```

4 Red nodes — inputs & field readers

Red nodes are pure sources: no geometry in, a .std_out you wire forward.

4.1 Constant inputs

InputValue (R) ,

InputInteger (R) ,

InputBoolean (R) ,

InputString (R) scalar/boolean/string constants.

InputVector (R) ,

InputRotation (R) vector/rotation constants.

InputMaterial (R) a material reference.

SceneTime (R) .std_out="Seconds" (or "Frame").

```
k = InputValue(tree, value=3.5, name="k")
v = InputVector(tree, value=[1, 0, 0], name="axis")
# use them downstream via .std_out:
fn_input = k.std_out
```

4.2 Per-element fields

Position (R) ,

Index (R) ,

InputNormal (R) ,

FaceArea (R) built-in fields.

NamedAttribute (R) read an attribute by name; data_type picks FLOAT/INT/BOOL/VECTOR/QUATERNION/COLOR. When fed to a Sample* node's Value input it evaluates in the sampled mesh's context.

CollectionInfo (R) geometry from a Blender collection.

4.3 Mesh topology readers

EdgeVertices (R) ,

VertexNeighbors (R) edge endpoints / vertex valence.

CornersOfEdge (R) ,

CornersOfFace (R) ,

EdgesOfCorner (R) ,

FaceOfCorner (R) ,

OffsetCornerInFace (R) corner/edge/face topology walks.

ShortestEdgePaths (R) shortest path along edges.

4.4 Rotation algebra (red)

InvertRotation (R) ,

RotateRotation (R) compose/invert rotations.

AlignRotationToVector (R) align an axis to a vector.

5 Blue nodes — compute & math

Blue nodes never touch geometry; they transform values. Every one sets `.std_out = outputs[0]`.

5.1 Scalar / vector math

MathNode (B) ShaderNodeMath; operation, inputs0..2.

VectorMath (B) ShaderNodeVectorMath; operation, inputs0/1, float_input (for SCALE).

BooleanMath (B) NOT/AND/OR/XOR/...; inputs0/1.

CompareNode (B) typed compare; data_type, operation, inputs0/1.

MapRange (B) remap a value; data_type for scalar/vector.

MixNode (B) mix float/vector/colour.

```
# (a > 0) AND (b == c)
gt = CompareNode(tree, operation="GREATER_THAN",
                 inputs0=a.std_out, inputs1=0)
eq = CompareNode(tree, operation="EQUAL",
                 inputs0=b.std_out, inputs1=c.std_out)
both = BooleanMath(tree, operation="AND",
                  inputs0=gt.std_out, inputs1=eq.std_out)
```

5.2 Vectors, matrices, conversions

SeparateXYZ (B) ,

CombineXYZ (B) (de)compose a vector.

CombineMatrix (B) build a 4×4 matrix from columns.

TransformPoint (B) apply a matrix to one vector.

ValueToString (B) ,

StringLength (B) ,

StringJoin (B) ,

SliceString (B) string ops.

5.3 Rotation algebra (blue)

EulerToRotation (B) ,

AxisAngleToRotation (B) ,

AxesToRotation (B) ,

QuaternionToRotation (B) build rotations.

RotateVector (B) ,

VectorRotate (B) rotate a vector.

5.4 Random & statistics

RandomValue (B) RNG; data_type (FLOAT/FLOAT_VECTOR/INT/BOOLEAN); kwargs min, max, seed, id, probability.

AttributeStatistic (B) min/max/mean/stddev over a domain.

EvaluateOnDomain (B) ,

EvaluateAtIndex (B) evaluate an attribute on a domain / at an index.

```
rng = RandomValue(tree, data_type="FLOAT_VECTOR",
                 min=Vector([-1,-1,0]),
                 max=Vector([1, 1,0]), seed=7)
offset = rng.std_out
```

6 Switches

Switch (B) boolean 2-way switch; input_type, switch, false, true.

IndexSwitch (B) N-way switch keyed by an INT. Add slots with `.new_item()`; `.slots[0]` is the *Index* socket, the *k*-th case is `.slots[k+1]`. `.std_out` is the Output.

MenuSwitch (B) string/menu-keyed switch.

```
sw = Switch(tree, input_type="VECTOR",
            switch=flag.std_out,
            false=a.std_out, true=b.std_out)

idx = IndexSwitch(tree, data_type="GEOMETRY",
                  index=key.std_out)
idx.new_item() # add a 3rd case
idx.add_item(geo_a.geometry_out)
idx.add_item(geo_b.geometry_out)
idx.add_item(geo_c.geometry_out)
result = idx.std_out
```

7 Structural / special nodes

GroupInput (N) ,

GroupOutput (N) the group's I/O nodes.

Reroute (N) typed pass-through wire; `ins=<socket>` feeds it inline. Exposes `.std_in/.std_out` and `.geometry_in/.geometry_out` (same socket), so it can sit in a geometry line too.

Frame (N) a visual container; `frame.add([n1, n2])` re-parents nodes into it.

Coord — a labelled grid of reroute nodes; index with `coord[(x,y)]`. Handy as a manual wiring lattice.

ExistingNodeGroup (N) wrap an already-created group tree by name (used by the XML importer).

```
frame = Frame(tree, name="setup")
frame.add([grid, set_pos, mat]) # group them visually

r = Reroute(tree, ins=value.std_out, location=(2, 1))
downstream_socket = r.std_out
```

8 Zones — loops & simulation

Zones come as a paired input/output node. The wrapper creates both, pairs them, and gives you `.geometry_in/.geometry_out` plus a `create_geometry_line(...)` that anchors a chain inside the loop body. Extra carry-through state is added with `.add_socket`.

8.1 RepeatZone — fixed iteration count

```
rep = RepeatZone(tree, iterations=10,
                 geometry=src.geometry_out)
# carry-through state lives on both the in and out
rep.add_socket("FLOAT", "energy")
# build the body between repeat_input and repeat_output
rep.create_geometry_line([step1, step2])
# the loop's iteration index:
i = rep.iteration
result = rep.geometry_out
```


8.2 ForEachZone — iterate over a domain

`.element` the per-iteration element
`(foreach_input.outputs["Element"])`.
`.index` the iteration index socket.
`.geometry_in / .geometry_out` loop in/out geometry.
`.add_socket(type, name, value, for_input=False)`
 carry-through inputs.

```
fe = ForEachZone(tree, domain="FACE",
                  geometry=mesh.geometry_out)
fe.create_geometry_line([transform], ins=fe.element)
out_geo = fe.geometry_out
```

8.3 Simulation — state across frames

Same API shape as `RepeatZone`: `.geometry_in/out`,
`.add_socket(type, name, value)`, and
`.create_geometry_line`. Use it for state that must
 persist between rendered frames (physics, growth,
 accumulators).

```
sim = Simulation(tree, geometry=seed.geometry_out)
sim.add_socket("VECTOR", "velocity", value=Vector())
sim.create_geometry_line([integrate, collide])
```

The lower-level `RepeatInput/RepeatOutput`,
`SimulationInput/SimulationOutput`,
`ForEachInput/ForEachOutput` wrappers exist but you
 rarely need them directly — prefer the zone wrappers
 above.

9 The make_function RPN builder

`make_function` is the single most-used helper. It builds
 a whole node group implementing arbitrary scalar/vec-
 tor/rotation math from *Reverse-Polish-Notation* strings,
 so you avoid hand-laying towers of `MathNode/VectorMath`.

```
xi = make_function(tree, name="xi",
                  functions={"xi": ["1,2,/","3,sqrt,2,/","0"]},
                  outputs=["xi"], vectors=["xi"],
                  location=(-8, -1), hide=True)

tree.links.new(src.outputs["phi"], xi.inputs["phi"])
result = xi.outputs["xi"]
```

Recipe:

- `inputs=[...]`, `outputs=[...]` are socket-name strings.
- Each name is typed by listing it in `scalars=[...]`,
`vectors=[...]` or `rotations=[...]`. **Outputs default
 to scalar** if you forget — a common source of link-type
 errors.
- `functions={name: "rpn"}` for a scalar output, or `{name:
 ["x","y","z"]}` to assemble a vector component-wise.
 A single string for a vector name is combined as one
 vector.
- `location` is in grid units.

Evaluation order. The parser pushes left-to-right and
 pops from the top, so `"a,b,sub"` computes $a - b$. A worked
 example: $T(p) = v_1 + \text{cmul}(p - v_4, r)/k$ is written

```
"p,v4,sub,rot,cmul,1,k,/,scale,v1,add"
```

9.1 RPN operator vocabulary

Scalar (`ShaderNodeMath`): `*` `/` `+` `-` `**` `%` `<` `>` `=`, `min` `max`
`atan2` `sin` `sinh` `cos` `cosh` `tan` `asin` `acos` `atan` `tanh` `lg`

`exp` `sqrt` `abs` `sgn` `round` `floor` `ceil` `frac`. Special: `=` sets
 the COMPARE epsilon to 0; `lg` is log base 10.

Vector (`ShaderNodeVectorMath`, names differ from
 scalars): `mul` `div` `add` `sub` `mod` `scale` (right operand
 scalar), `cross` `dot` (scalar), `length` (scalar), `vfloor`
`normalize`.

Rotations: `rot` (apply Euler XYZ to a vec-
 tor), `axis_rot` (axis+angle→rotation), `rot2euler`
`axis_angle_euler` `rot_vec` `inv_rot`.

Booleans: `not` and `or`.

Complex (a VECTOR is treated as $(\text{Re}, \text{Im}, |z|)$,
 dispatched to `ComplexMathNode`): `cadd` `csub` `cmul` `cdiv`
`cscale` (right operand scalar), `cconj` `cabs`.

10 Wiring helpers

`create_geometry_line(tree, [n1,n2,...], ins, out)` –
 chains the geometry ports of a list of green nodes left-to-
 right; optional `ins` source and `out` target.

`layout(tree, mode="Sugiyama")` – auto-layout; run
once at the end. Never run it on a hand-positioned tree —
 it undoes your work.

`add_locations(loc1, loc2)` – element-wise tuple add;
 handy when shifting a block of nodes.

11 Custom group nodes — NodeGroup

When you need an inner sub-tree (so the parent shows
 one tidy node), subclass `NodeGroup`. The base class cre-
 ates the group, its `GroupInput/GroupOutput`, and the re-
 quested sockets; your subclass fills the inner graph in
`fill_group_with_node`.

```
class MyThing(NodeGroup):
    def __init__(self, tree, **kwargs):
        super().__init__(tree,
                          inputs={"x": "VECTOR", "k": "VALUE"},
                          outputs={"y": "VECTOR"},
                          name="MyThing", **kwargs)
        self.std_out = self.node.outputs["y"]

    def fill_group_with_node(self, tree, **kwargs):
        # 'tree' is the *inner* tree (self.group_tree)
        fn = make_function(tree, name="core",
                          functions={"y": "x,k,scale"},
                          inputs=["x","k"], outputs=["y"],
                          vectors=["x","y"], scalars=["k"])
        tree.links.new(self.group_inputs.outputs["x"],
                       fn.inputs["x"])
        tree.links.new(self.group_inputs.outputs["k"],
                       fn.inputs["k"])
        tree.links.new(fn.outputs["y"],
                       self.group_outputs.inputs["y"])
```

11.1 Catalogue of ready-made groups (NG)

TransformPositionNode (NG)

`position*scale*rotation+location` (and its inverse
 via an Undo Transformation flag).

BevelNode (NG) ,

BeveledCubeNode (NG) ,

BevelFaces (NG) bevel helpers.

ComplexMathNode (NG) complex arithmetic over a
 VECTOR ($X = \text{Re}$, $Y = \text{Im}$, $Z = |z|$): `ADD/SUB/MUL/DIV/`
`SCALE/CONJ/ABS`.

CoxeterReflectionNode (NG) Coxeter-group reflec-
 tion.

Rotation (G) ,

Matrix (G) ,
Transpose (G) ,
LinearMap (G) ,
ProjectionMap (G) linear-algebra helpers (custom green groups).
InsideConvexHull (G) ,
InsideConvexHull3D (G) ,
InsidePolygon (G) point-in-region tests; `.std_out = "Is Inside"`.
CycleNode (NG) ,
SimpleRubiksCubeNode (NG) ,
TranslateToCenterNode (NG) ,
SlicerNode (NG) ,
UnfoldMeshNode (NG) ,
PolyhedronViewNode (NG) ,
ShowNormalsNode (NG) ,
E8Node (G) domain-specific helpers for particular videos.

12 Building a scene: GeometryNodesModifier

Scenes subclass `GeometryNodesModifier` and override `create_node`. The base class creates the tree, a group output (and optionally a group input), runs your `create_node`, and — unless `automatic_layout=False` — lays the result out.

```
class MyModifier(GeometryNodesModifier):
    def __init__(self, n=100, **kwargs):
        self.n = n
        super().__init__(name="MyModifier",
                         automatic_layout=False, **kwargs)

    def create_node(self, tree, **kwargs):
        out = tree.nodes.get("Group Output")
        grid = Grid(tree, location=(-6, 0))
        spos = SetPosition(tree, location=(-4, 0),
                           position=p_fn.outputs["p"])
        mat = SetMaterial(tree, location=(-2, 0),
                          material=self.materials[0])
        create_geometry_line(tree, [grid, spos, mat],
                              out=out.inputs["Geometry"])
```

What the base class hands you:

- `self.group_outputs` (the IO node) and `self.group_output` (same node, by name) with one "Geometry" output socket.
- `self.group_inputs` when `group_input=True`.
- `self.tree` / `self.nodes` for direct access.

Materials created inside `create_node` should be appended to `self.materials`; the framework adds them to the host object's material slots.

13 XML import path

`create_from_xml(tree, filename=...)` reconstructs a node tree that was exported from Blender as XML, dispatching each node through `Node.from_attributes`. This is the round-trip path: iterate a graph visually in Blender, export `tmp.xml`, then port it back to Python.

```
create_from_xml(tree, filename="my_graph.xml")
```

`IndexSwitch` and `MenuSwitch` have an `add_new_item_from_xml(...)` path used *only* by the

importer — in normal code use `.new_item()` / `.add_item()` instead.

14 Footguns actually hit

1. **SeparateGeometry(geometry=src)** ignores the kwarg. Always `tree.links.new(src, sep.geometry_in)`.
2. **SampleNearestSurface** uses "Mesh" not "Geometry". The `mesh=` kwarg links the wrong socket; use `tree.links.new(src, snn.node.inputs["Mesh"])`.
3. **IndexSwitch slot indexing**. `.slots[0]` is the Index input; the k -th case is `.slots[k+1]`. Ships with 2 cases; call `.new_item()` for each extra one *before* indexing it.
4. **Automatic layout eats hand-placed nodes**. `automatic_layout=False` is mandatory for any graph you want to read.
5. **Location units are grid, not pixels**. $(1,0) = (200,0)$ in Blender.
6. **make_function order is RPN, popped from the top**. "a,b,sub" produces $a - b$; "v4,v1,sub" is $v_4 - v_1$.
7. **make_function outputs default to scalar**. List a name in `vectors=...` or `rotations=...` or you get confusing socket-type errors at link time.
8. **Two classes are defined twice** in `nodes.py` (`Switch`, `TransformPositionNode`, `BeveledCubeNode`): the *later* definition wins.

Generated from `geometry_nodes/nodes.py`; cross-checked against `geometry_nodes_modifier.py` and `learning_from_claude/geometrynodes.md`.